

```
-- PostgreSQL DDL Script
-- Sistema de Gerenciamento de Vendas e Compras
-- Gerado em: 05/11/2025

-- =====
-- TABELA: cliente
-- Descrição: Cadastro de clientes
-- =====

CREATE TABLE cliente (
    id_cliente SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    sobrenome VARCHAR(100) NOT NULL,
    cpf VARCHAR(14) UNIQUE NOT NULL,
    endereco TEXT,
    telefone VARCHAR(20),
    email VARCHAR(255),
    ativo BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_cpf_formato CHECK (cpf ~ '^\\d{3}\\.\\d{3}\\.\\d{3}-\\d{2}$' OR cpf ~ '^\\d{11}$')
);

-- Índices
CREATE INDEX idx_cliente_cpf ON cliente(cpf);
CREATE INDEX idx_cliente_nome ON cliente(nome, sobrenome);
CREATE INDEX idx_cliente_ativo ON cliente(ativo);

-- Comentários
COMMENT ON TABLE cliente IS 'Cadastro de clientes';
COMMENT ON COLUMN cliente.cpf IS 'CPF do cliente (formato: XXX.XXX.XXX-XX ou apenas números)';
COMMENT ON COLUMN cliente.ativo IS 'Indica se o cliente está ativo no sistema';

-- =====
-- TABELA: fornecedor
-- Descrição: Cadastro de fornecedores
-- =====

CREATE TABLE fornecedor (
    id_fornecedor SERIAL PRIMARY KEY,
    nome_empresa VARCHAR(200) NOT NULL,
    nome_contato VARCHAR(100),
    cnpj VARCHAR(18) UNIQUE NOT NULL,
    telefone VARCHAR(20),
    email VARCHAR(255),
    endereco TEXT,
    ativo BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_cnpj_formato CHECK (cnpj ~ '^\\d{2}\\.\\d{3}\\.\\d{3}/\\d{4}-\\d{2}$' OR cnpj ~ '^\\d{14}$')
);

-- Índices
CREATE INDEX idx_fornecedor_cnpj ON fornecedor(cnpj);
CREATE INDEX idx_fornecedor_nome ON fornecedor(nome_empresa);
CREATE INDEX idx_fornecedor_ativo ON fornecedor(ativo);

-- Comentários
COMMENT ON TABLE fornecedor IS 'Cadastro de fornecedores';
COMMENT ON COLUMN fornecedor.cnpj IS 'CNPJ do fornecedor (formato: XX.XXX.XXX/XXXX-XX ou apenas números)';

-- =====
-- TABELA: produto
-- Descrição: Catálogo de produtos
-- =====

CREATE TABLE produto (
    id_produto SERIAL PRIMARY KEY,
    nome_produto VARCHAR(200) NOT NULL,
    descricao TEXT,
    categoria VARCHAR(100),
    preco_venda NUMERIC(10, 2) NOT NULL,
    preco_custo NUMERIC(10, 2),
    quantidade_estoque INTEGER DEFAULT 0,
    estoque_minimo INTEGER DEFAULT 0,
    codigo_barras VARCHAR(50) UNIQUE,
    ativo BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_preco_venda CHECK (preco_venda >= 0),
    CONSTRAINT chk_preco_custo CHECK (preco_custo IS NULL OR preco_custo >= 0),
    CONSTRAINT chk_quantidade_estoque CHECK (quantidade_estoque >= 0),
    CONSTRAINT chk_estoque_minimo CHECK (estoque_minimo >= 0)
);

-- Índices
CREATE INDEX idx_produto_nome ON produto(nome_produto);
CREATE INDEX idx_produto_categoria ON produto(categoria);
CREATE INDEX idx_produto_codigo_barras ON produto(codigo_barras);
```

```
CREATE INDEX idx_produto_ativo ON produto(ativo);
CREATE INDEX idx_produto_estoque ON produto(quantidade_estoque);

-- Comentários
COMMENT ON TABLE produto IS 'Catálogo de produtos';
COMMENT ON COLUMN produto.preco_venda IS 'Preço de venda ao cliente';
COMMENT ON COLUMN produto.preco_custo IS 'Preço de custo/compra do produto';
COMMENT ON COLUMN produto.quantidade_estoque IS 'Quantidade atual em estoque';
COMMENT ON COLUMN produto.estoque_minimo IS 'Quantidade mínima de estoque para alerta';

-- =====
-- TABELA: venda
-- Descrição: Registro de vendas realizadas
-- =====

CREATE TABLE venda (
    id_venda SERIAL PRIMARY KEY,
    cliente_id INTEGER NOT NULL,
    data_venda TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    valor_subtotal NUMERIC(12, 2) NOT NULL DEFAULT 0,
    valor_desconto NUMERIC(12, 2) DEFAULT 0,
    valor_total NUMERIC(12, 2) NOT NULL DEFAULT 0,
    forma_pagamento VARCHAR(50),
    status VARCHAR(20) DEFAULT 'CONCLUIDA',
    observacoes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_venda_cliente
        FOREIGN KEY (cliente_id)
        REFERENCES cliente(id_cliente)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT chk_valor_subtotal CHECK (valor_subtotal >= 0),
    CONSTRAINT chk_valor_desconto CHECK (valor_desconto >= 0),
    CONSTRAINT chk_valor_total CHECK (valor_total >= 0),
    CONSTRAINT chk_status_venda CHECK (status IN ('PENDENTE', 'CONCLUIDA', 'CANCELADA', 'DEVOLVIDA')),
    CONSTRAINT chk_forma_pagamento CHECK (
        forma_pagamento IS NULL OR
        forma_pagamento IN ('DINHEIRO', 'CARTAO_CREDITO', 'CARTAO_DEBITO', 'PIX', 'BOLETO', 'TRANSFERENCIA')
    )
);

-- Índices
CREATE INDEX idx_venda_cliente ON venda(cliente_id);
CREATE INDEX idx_venda_data ON venda(data_venda);
CREATE INDEX idx_venda_status ON venda(status);
CREATE INDEX idx_venda_forma_pagamento ON venda(forma_pagamento);

-- Comentários
COMMENT ON TABLE venda IS 'Registro de vendas realizadas';
COMMENT ON COLUMN venda.valor_subtotal IS 'Soma dos itens antes do desconto';
COMMENT ON COLUMN venda.valor_desconto IS 'Valor de desconto aplicado';
COMMENT ON COLUMN venda.valor_total IS 'Valor final da venda (subtotal - desconto)';

-- =====
-- TABELA: item_venda
-- Descrição: Itens de cada venda
-- =====

CREATE TABLE item_venda (
    id_item_venda SERIAL PRIMARY KEY,
    venda_id INTEGER NOT NULL,
    produto_id INTEGER NOT NULL,
    quantidade INTEGER NOT NULL,
    preco_unitario NUMERIC(10, 2) NOT NULL,
    desconto_item NUMERIC(10, 2) DEFAULT 0,
    valor_total NUMERIC(12, 2) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_item_venda_venda
        FOREIGN KEY (venda_id)
        REFERENCES venda(id_venda)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_item_venda_produto
        FOREIGN KEY (produto_id)
        REFERENCES produto(id_produto)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT chk_quantidade CHECK (quantidade > 0),
    CONSTRAINT chk_preco_unitario CHECK (preco_unitario >= 0),
    CONSTRAINT chk_desconto_item CHECK (desconto_item >= 0),
    CONSTRAINT chk_valor_total_item CHECK (valor_total >= 0)
);

-- Índices
CREATE INDEX idx_item_venda_venda ON item_venda(venda_id);
CREATE INDEX idx_item_venda_produto ON item_venda(produto_id);

-- Comentários
```

```
COMMENT ON TABLE item_venda IS 'Itens individuais de cada venda';
COMMENT ON COLUMN item_venda.valor_total IS 'Valor total do item (quantidade * preco_unitario - desconto_item)';

-- =====
-- TABELA: compra
-- Descrição: Registro de compras de fornecedores
-- =====

CREATE TABLE compra (
    id_compra SERIAL PRIMARY KEY,
    fornecedor_id INTEGER NOT NULL,
    data_compra TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data_entrega DATE,
    valor_subtotal NUMERIC(12, 2) NOT NULL DEFAULT 0,
    valor_frete NUMERIC(12, 2) DEFAULT 0,
    valor_desconto NUMERIC(12, 2) DEFAULT 0,
    valor_total NUMERIC(12, 2) NOT NULL DEFAULT 0,
    status VARCHAR(20) DEFAULT 'PENDENTE',
    numero_nota_fiscal VARCHAR(50),
    observacoes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_compra_fornecedor
        FOREIGN KEY (fornecedor_id)
        REFERENCES fornecedor(id_fornecedor)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT chk_valor_subtotal_compra CHECK (valor_subtotal >= 0),
    CONSTRAINT chk_valor_frete CHECK (valor_frete >= 0),
    CONSTRAINT chk_valor_desconto_compra CHECK (valor_desconto >= 0),
    CONSTRAINT chk_valor_total_compra CHECK (valor_total >= 0),
    CONSTRAINT chk_status_compra CHECK (status IN ('PENDENTE', 'RECEBIDA', 'CANCELADA', 'DEVOLVIDA'))
);

-- Índices
CREATE INDEX idx_compra_fornecedor ON compra(fornecedor_id);
CREATE INDEX idx_compra_data ON compra(data_compra);
CREATE INDEX idx_compra_status ON compra(status);
CREATE INDEX idx_compra_nota_fiscal ON compra(numero_nota_fiscal);

-- Comentários
COMMENT ON TABLE compra IS 'Registro de compras de fornecedores';
COMMENT ON COLUMN compra.valor_total IS 'Valor final da compra (subtotal + frete - desconto)';

-- =====
-- TABELA: item_compra
-- Descrição: Itens de cada compra
-- =====

CREATE TABLE item_compra (
    id_item_compra SERIAL PRIMARY KEY,
    compra_id INTEGER NOT NULL,
    produto_id INTEGER NOT NULL,
    quantidade INTEGER NOT NULL,
    preco_unitario NUMERIC(10, 2) NOT NULL,
    desconto_item NUMERIC(10, 2) DEFAULT 0,
    valor_total NUMERIC(12, 2) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_item_compra_compra
        FOREIGN KEY (compra_id)
        REFERENCES compra(id_compra)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_item_compra_produto
        FOREIGN KEY (produto_id)
        REFERENCES produto(id_produto)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT chk_quantidade_compra CHECK (quantidade > 0),
    CONSTRAINT chk_preco_unitario_compra CHECK (preco_unitario >= 0),
    CONSTRAINT chk_desconto_item_compra CHECK (desconto_item >= 0),
    CONSTRAINT chk_valor_total_item_compra CHECK (valor_total >= 0)
);

-- Índices
CREATE INDEX idx_item_compra_compra ON item_compra(compra_id);
CREATE INDEX idx_item_compra_produto ON item_compra(produto_id);

-- Comentários
COMMENT ON TABLE item_compra IS 'Itens individuais de cada compra';
COMMENT ON COLUMN item_compra.valor_total IS 'Valor total do item (quantidade * preco_unitario - desconto_item)';

-- =====
-- FUNÇÕES E TRIGGERS
-- =====

-- Função para atualizar timestamp automaticamente
CREATE OR REPLACE FUNCTION atualizar_updated_at()
RETURNS TRIGGER AS $$
```

```

BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Triggers para atualização automática de updated_at
CREATE TRIGGER trigger_cliente_updated_at
    BEFORE UPDATE ON cliente
    FOR EACH ROW
    EXECUTE FUNCTION atualizar_updated_at();

CREATE TRIGGER trigger_fornecedor_updated_at
    BEFORE UPDATE ON fornecedor
    FOR EACH ROW
    EXECUTE FUNCTION atualizar_updated_at();

CREATE TRIGGER trigger_produto_updated_at
    BEFORE UPDATE ON produto
    FOR EACH ROW
    EXECUTE FUNCTION atualizar_updated_at();

CREATE TRIGGER trigger_venda_updated_at
    BEFORE UPDATE ON venda
    FOR EACH ROW
    EXECUTE FUNCTION atualizar_updated_at();

CREATE TRIGGER trigger_compra_updated_at
    BEFORE UPDATE ON compra
    FOR EACH ROW
    EXECUTE FUNCTION atualizar_updated_at();

-- Função para calcular valor total do item de venda
CREATE OR REPLACE FUNCTION calcular_valor_item_venda()
RETURNS TRIGGER AS $$
BEGIN
    NEW.valor_total = (NEW.quantidade * NEW.preco_unitario) - COALESCE(NEW.desconto_item, 0);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger para calcular automaticamente valor total do item de venda
CREATE TRIGGER trigger_calcular_valor_item_venda
    BEFORE INSERT OR UPDATE ON item_venda
    FOR EACH ROW
    EXECUTE FUNCTION calcular_valor_item_venda();

-- Função para calcular valor total do item de compra
CREATE OR REPLACE FUNCTION calcular_valor_item_compra()
RETURNS TRIGGER AS $$
BEGIN
    NEW.valor_total = (NEW.quantidade * NEW.preco_unitario) - COALESCE(NEW.desconto_item, 0);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger para calcular automaticamente valor total do item de compra
CREATE TRIGGER trigger_calcular_valor_item_compra
    BEFORE INSERT OR UPDATE ON item_compra
    FOR EACH ROW
    EXECUTE FUNCTION calcular_valor_item_compra();

-- Função para atualizar estoque após venda
CREATE OR REPLACE FUNCTION atualizar_estoque_venda()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE produto
        SET quantidade_estoque = quantidade_estoque - NEW.quantidade
        WHERE id_produto = NEW.produto_id;
    ELSIF TG_OP = 'DELETE' THEN
        UPDATE produto
        SET quantidade_estoque = quantidade_estoque + OLD.quantidade
        WHERE id_produto = OLD.produto_id;
    ELSIF TG_OP = 'UPDATE' THEN
        UPDATE produto
        SET quantidade_estoque = quantidade_estoque + OLD.quantidade - NEW.quantidade
        WHERE id_produto = NEW.produto_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger para atualizar estoque automaticamente em vendas
CREATE TRIGGER trigger_atualizar_estoque_venda
    AFTER INSERT OR UPDATE OR DELETE ON item_venda
    FOR EACH ROW

```

```

EXECUTE FUNCTION atualizar_estoque_venda();

-- Função para atualizar estoque após compra
CREATE OR REPLACE FUNCTION atualizar_estoque_compra()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE produto
        SET quantidade_estoque = quantidade_estoque + NEW.quantidade
        WHERE id_produto = NEW.produto_id;
    ELSIF TG_OP = 'DELETE' THEN
        UPDATE produto
        SET quantidade_estoque = quantidade_estoque - OLD.quantidade
        WHERE id_produto = OLD.produto_id;
    ELSIF TG_OP = 'UPDATE' THEN
        UPDATE produto
        SET quantidade_estoque = quantidade_estoque - OLD.quantidade + NEW.quantidade
        WHERE id_produto = NEW.produto_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger para atualizar estoque automaticamente em compras
CREATE TRIGGER trigger_atualizar_estoque_compra
AFTER INSERT OR UPDATE OR DELETE ON item_compra
FOR EACH ROW
EXECUTE FUNCTION atualizar_estoque_compra();

-- Função para recalcular totais da venda
CREATE OR REPLACE FUNCTION recalcular_totais_venda()
RETURNS TRIGGER AS $$
DECLARE
    v_subtotal NUMERIC(12, 2);
BEGIN
    SELECT COALESCE(SUM(valor_total), 0) INTO v_subtotal
    FROM item_venda
    WHERE venda_id = COALESCE(NEW.venda_id, OLD.venda_id);

    UPDATE venda
    SET valor_subtotal = v_subtotal,
        valor_total = v_subtotal - COALESCE(valor_desconto, 0)
    WHERE id_venda = COALESCE(NEW.venda_id, OLD.venda_id);

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger para recalcular totais da venda
CREATE TRIGGER trigger_recalcular_totais_venda
AFTER INSERT OR UPDATE OR DELETE ON item_venda
FOR EACH ROW
EXECUTE FUNCTION recalcular_totais_venda();

-- Função para recalcular totais da compra
CREATE OR REPLACE FUNCTION recalcular_totais_compra()
RETURNS TRIGGER AS $$
DECLARE
    v_subtotal NUMERIC(12, 2);
BEGIN
    SELECT COALESCE(SUM(valor_total), 0) INTO v_subtotal
    FROM item_compra
    WHERE compra_id = COALESCE(NEW.compra_id, OLD.compra_id);

    UPDATE compra
    SET valor_subtotal = v_subtotal,
        valor_total = v_subtotal + COALESCE(valor_frete, 0) - COALESCE(valor_desconto, 0)
    WHERE id_compra = COALESCE(NEW.compra_id, OLD.compra_id);

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger para recalcular totais da compra
CREATE TRIGGER trigger_recalcular_totais_compra
AFTER INSERT OR UPDATE OR DELETE ON item_compra
FOR EACH ROW
EXECUTE FUNCTION recalcular_totais_compra();

-- =====
-- VIEWS ÚTEIS
-- =====

-- View para produtos com estoque baixo
CREATE OR REPLACE VIEW vw_produtos_estoque_baixo AS
SELECT
    id_produto,

```

```

        nome_produto,
        categoria,
        quantidade_estoque,
        estoque_minimo,
        preco_venda,
        (estoque_minimo - quantidade_estoque) AS quantidade_repor
FROM produto
WHERE quantidade_estoque <= estoque_minimo
      AND ativo = TRUE
ORDER BY (estoque_minimo - quantidade_estoque) DESC;

COMMENT ON VIEW vw_produtos_estoque_baixo IS 'Produtos com estoque abaixo do mínimo';

-- View para vendas completas
CREATE OR REPLACE VIEW vw_vendas_completas AS
SELECT
    v.id_venda,
    v.data_venda,
    c.nome || ' ' || c.sobrenome AS nome_cliente,
    c.cpf,
    v.valor_subtotal,
    v.valor_desconto,
    v.valor_total,
    v.forma_pagamento,
    v.status,
    COUNT(iv.id_item_venda) AS total_itens
FROM venda v
INNER JOIN cliente c ON v.cliente_id = c.id_cliente
LEFT JOIN item_venda iv ON v.id_venda = iv.venda_id
GROUP BY v.id_venda, v.data_venda, c.nome, c.sobrenome, c.cpf,
         v.valor_subtotal, v.valor_desconto, v.valor_total,
         v.forma_pagamento, v.status
ORDER BY v.data_venda DESC;

COMMENT ON VIEW vw_vendas_completas IS 'Visão completa das vendas com informações do cliente';

-- View para compras completas
CREATE OR REPLACE VIEW vw_compras_completas AS
SELECT
    c.id_compra,
    c.data_compra,
    c.data_entrega,
    f.nome_empresa AS fornecedor,
    f.cnpj,
    c.valor_subtotal,
    c.valor_frete,
    c.valor_desconto,
    c.valor_total,
    c.status,
    c.numero_nota_fiscal,
    COUNT(ic.id_item_compra) AS total_itens
FROM compra c
INNER JOIN fornecedor f ON c.fornecedor_id = f.id_fornecedor
LEFT JOIN item_compra ic ON c.id_compra = ic.compra_id
GROUP BY c.id_compra, c.data_compra, c.data_entrega, f.nome_empresa, f.cnpj,
         c.valor_subtotal, c.valor_frete, c.valor_desconto, c.valor_total,
         c.status, c.numero_nota_fiscal
ORDER BY c.data_compra DESC;

COMMENT ON VIEW vw_compras_completas IS 'Visão completa das compras com informações do fornecedor';

-- View para análise de vendas por produto
CREATE OR REPLACE VIEW vw_analise_vendas_produto AS
SELECT
    p.id_produto,
    p.nome_produto,
    p.categoria,
    COUNT(DISTINCT iv.venda_id) AS total_vendas,
    SUM(iv.quantidade) AS quantidade_total_vendida,
    SUM(iv.valor_total) AS receita_total,
    AVG(iv.preco_unitario) AS preco_medio_venda,
    p.quantidade_estoque AS estoque_atual
FROM produto p
LEFT JOIN item_venda iv ON p.id_produto = iv.produto_id
GROUP BY p.id_produto, p.nome_produto, p.categoria, p.quantidade_estoque
ORDER BY receita_total DESC NULLS LAST;

COMMENT ON VIEW vw_analise_vendas_produto IS 'Análise de desempenho de vendas por produto';

-- View para produtos sem movimentação
CREATE OR REPLACE VIEW vw_produtos_sem_movimentacao AS
SELECT
    p.id_produto,
    p.nome_produto,
    p.categoria,
    p.quantidade_estoque,
    p.preco_venda,

```

```

    p.created_at AS data_cadastro
FROM produto p
LEFT JOIN item_venda iv ON p.id_produto = iv.produto_id
LEFT JOIN item_compra ic ON p.id_produto = ic.produto_id
WHERE iv.id_item_venda IS NULL
      AND ic.id_item_compra IS NULL
      AND p.ativo = TRUE
ORDER BY p.created_at;

COMMENT ON VIEW vw_produtos_sem_movimentacao IS 'Produtos que nunca foram vendidos ou comprados';

-- =====
-- FIM DO SCRIPT
-- =====
```